

Table Extraction Code — Line-by-Line Explanation

A walkthrough of the Python loop that filters `TableItem` objects from a document parser's structured output and persists each table's four representations (rows, CSV, Markdown, HTML) to disk plus a JSON manifest.

Line-by-Line Explanation

```
table_records = []
```

Creates an empty list to collect one dictionary per table found. This becomes your master manifest — everything you will eventually dump to JSON. Like `var tableRecords = new List<Dictionary<string, object>>();` in C#.

```
table_counter = 0
```

A running count that increments across all pages. Gives each table a global ID (table 1, 2, 3, ...) independent of which page it lives on.

```
if result.items:
```

Guard against a null or empty parser result. If the parser returned nothing, skip the entire loop. Equivalent to `if (result?.Items != null)`.

```
    for page in result.items.pages:
```

Outer loop — iterate every page the parser produced (24 pages in your test document).

```
        page_number = page.page_number
```

Grab the 1-based page number for use in filenames and the JSON record.

```
        if not getattr(page, "success", True):
            continue
```

Skip pages the parser failed on. `getattr` defaults to `True` if the attribute is missing (assume success). In your data every page has `success=True`, so nothing is skipped.

```
            for item_index, item in enumerate(page.items, start=1):
```

Inner loop — walk every item on this page (headings, paragraphs, tables, footers, images). `enumerate(..., start=1)` gives a 1-based position within the page.

```
                if getattr(item, "type", None) != "table":
                    continue
```

The critical filter. Skip anything that is not a table — headings, text, footers, code blocks, images. Only `TableItem` objects survive past this line.

```
                table_counter += 1
```

Increment the global table counter. First table encountered becomes table 1, next becomes 2, and so on across all pages.

```

rows = getattr(item, "rows", []) or []
csv_text = getattr(item, "csv", "") or ""
markdown_text = getattr(item, "md", "") or ""
html_text = getattr(item, "html", "") or ""

```

Extract the four pre-built representations off the `TableItem`. Both `getattr(..., default)` and `or default` guard against missing attributes and `None` values respectively. You end up with a list of rows plus three string formats.

```

table_record = {
    "table_number": table_counter,
    "page_number": page_number,
    "item_index": item_index,
    "rows": rows,
    "csv": csv_text,
    "markdown": markdown_text,
    "html": html_text,
}

```

Build a dictionary — one record for this table — capturing where it came from and all four representations. This is the row that goes into your master list.

```

table_records.append(table_record)

```

Add the record to the master list. By loop end, `table_records` holds one dict per table.

```

base_name = (
    f"page_{page_number:03d}"
    f"_table_{table_counter:03d}"
)

```

Construct a filename stem like `page_001_table_001`. The `:03d` zero-pads to 3 digits so files sort correctly in a directory listing (`page_001` before `page_010`).

```

csv_path = TABLE_DIR / f"{base_name}.csv"
markdown_path = TABLE_DIR / f"{base_name}.md"
html_path = TABLE_DIR / f"{base_name}.html"

```

Build three `Path` objects for the three output files. `TABLE_DIR /` is `pathlib` path joining — equivalent to `Path.Combine(tableDir, $"{baseName}.csv")` in .NET.

```

if csv_text:
    csv_path.write_text(csv_text, encoding="utf-8")

```

Write the CSV file — but only if there is actual content. Empty strings are falsy in Python, so this skips writes for tables where CSV was not produced.

```

if markdown_text:
    markdown_path.write_text(markdown_text, encoding="utf-8")

```

Same pattern for Markdown. Skip empty writes.

```

if html_text:
    html_path.write_text(html_text, encoding="utf-8")

```

Same pattern for HTML.

```

try:
    if csv_text:
        dataframe = pd.read_csv(io.StringIO(csv_text))
    elif rows:
        dataframe = pd.DataFrame(rows[1:], columns=rows[0])
    else:
        dataframe = pd.DataFrame()

```

Build a pandas DataFrame for preview. Prefer parsing the CSV string; fall back to constructing directly from rows (with rows[0] as headers, rows[1:] as data); last resort is an empty DataFrame.

```

print(f"\nTable {table_counter} found on page {page_number}")
print(dataframe.head())

```

Print a preview to the console — the first five rows of each table. Handy for eyeballing that extraction worked.

```

except Exception as error:
    print(f"Could not create DataFrame for table {table_counter}: {error}")

```

If pandas chokes on malformed CSV (say, a table with `<br/>` inside cells breaking the parser), log it and keep going. The disk files are already written before this try/except, so a preview failure does not lose data.

```

table_records_path = OUTPUT_DIR / "table_records.json"

```

Path for the master JSON manifest, at the top level of your output directory.

```

with table_records_path.open("w", encoding="utf-8") as file:
    json.dump(table_records, file, indent=2,
              ensure_ascii=False, default=str)

```

Write the entire table_records list as one JSON file. indent=2 pretty-prints for readability. ensure_ascii=False keeps non-ASCII characters (INR, á, é) as-is instead of escaping them. default=str handles any object types JSON does not know how to serialize by calling str() on them — a safety net.

```

print("\nTotal tables found:", len(table_records))
print("Table records saved:", table_records_path)

```

Final summary: how many tables were extracted and where the manifest was saved.

What the Code Does — Summary

This code walks the parser's structured output page by page, filters down to just the table items (ignoring headings, paragraphs, footers, and images), and for each table it captures four representations — a raw list of rows, a CSV string, a Markdown pipe-table, and an HTML table with merged-cell structure. Every table gets written to disk as three separate files (.csv, .md, .html) using a sortable naming convention (page_NNN_table_NNN), and a master JSON manifest (table_records.json) is written at the end containing all four representations plus provenance (page number, per-page position, global table number) for every table found.

Along the way it prints a pandas preview of each table so you can visually confirm extraction quality. It is a **filtering-and-serialization** step, not a text-to-table conversion — the parser already identified rows and columns upstream; this code just persists that structured output in multiple formats for downstream consumption by a RAG chunker, an analyst opening files in Excel, or an LLM reading Markdown.

The Two Stages at a Glance

Stage 1 — Parser (upstream, already done): reads the PDF, runs layout analysis, detects headings vs. tables vs. images, reconstructs table grids (rows, columns, headers, merged cells), and emits typed items with four pre-built representations per table.

Stage 2 — This code: walks the typed items, filters to tables only, and persists the four representations to disk plus a JSON manifest for downstream stages.

Where It Fits in the RAG Pipeline

